



Practical Malware Analysis & Triage

Malware Analysis Report

Agent Tesla-Style Malware Loader Analysis

April 2026 | Brianna Wandt | v1.0



Table of Contents

Table of Contents.....	2
Executive Summary.....	4
High-Level Technical Summary.....	6
Malware Architecture Overview.....	8
Malware Execution Flow.....	9
Key Findings.....	11
Basic Static Analysis.....	13
Section Header Anomaly Summary.....	13
Advanced Static Analysis.....	16
Runtime Unpacking & Memory Analysis.....	22
Embedded Resource & String Analysis.....	25
Basic Dynamic Analysis.....	27
Advanced Dynamic Analysis.....	28
Embedded Resources & Runtime Components.....	29
Networking & Communication Indicators.....	32
Cryptographic & Serialization Indicators.....	34
System Interaction & Environment Awareness.....	36
Injection & Evasion Indicators.....	38
Analysis Limitations.....	43
Defense / Anti-Analysis Techniques Identified.....	44
Packing and Obfuscation Analysis (DIE).....	45
Defense / Anti-Analysis Techniques Identified.....	46
Key Runtime & Embedded Components.....	47
Indicators of Compromise.....	48
Network Indicators.....	49
Host-based Indicators.....	51
Language Identification.....	53
Rules & Signatures.....	54
MITRE ATT&CK Mapping.....	55
Appendices.....	57
A. Yara Rules.....	57



Executive Summary

SHA256 hash	2caae88b894455be6fd124fb2bff81452c7f5249d3afda7daa5567740300b88a
-------------	--

The analyzed sample (Agenttesla.exe) is a heavily obfuscated 64-bit .NET malware loader exhibiting characteristics commonly associated with Agent Tesla-style information stealing and remote communication malware. Static analysis and limited dynamic analysis identified extensive use of packing, virtualization-based obfuscation, runtime assembly loading, protected or encoded resource handling, and anti-analysis protections intended to conceal functionality and disrupt reverse engineering efforts. Although definitive attribution to a specific malware family could not be conclusively established due to extensive obfuscation and limited observable runtime behavior, the recovered artifacts and behavioral indicators are consistent with techniques commonly observed in protected Agent Tesla-style malware loaders.

The executable embeds multiple compressed .NET dependencies directly within the binary through the Costura framework and dynamically loads required assemblies into memory during execution. Analysis identified references to indicators associated with HTTP communication, SMTP functionality, JSON serialization, cryptographic operations, registry interaction, and asynchronous networking methods associated with remote communication and data transfer.

Additional functionality identified during analysis includes:

- HTTP GET and POST communication routines
- encrypted or serialized configuration handling
- Base64 decoding and cryptographic stream processing
- asynchronous download and upload functionality
- registry interaction and environment awareness
- process injection indicators
- runtime assembly unpacking
- anti-debugging and self-termination behavior

The malware contains multiple indicators of advanced defense evasion mechanisms, including malformed Intermediate Language (IL), virtualization-based protections associated with the XVM runtime, high-entropy packed sections, and dynamic runtime execution techniques that prevented successful full decompilation within dnSpy and ILSpy.



Dynamic execution within the FLARE VM analysis environment produced minimal observable behavior. The sample terminated shortly after execution without generating significant filesystem, registry, or network artifacts. This behavior strongly suggests the presence of sandbox or virtual machine detection mechanisms intended to suppress execution in monitored or virtualized environments.

Although complete behavioral execution could not be fully observed due to heavy obfuscation and anti-analysis protections, the combined static indicators strongly support the conclusion that the combined static and runtime indicators suggest the sample was designed to operate as a stealth-oriented malware loader supporting protected in-memory execution, remote communication functionality, and encoded or serialized runtime data handling.



High-Level Technical Summary

The analyzed sample is a heavily obfuscated 64-bit .NET executable exhibiting characteristics commonly associated with Agent Tesla-style information stealing malware. Static analysis identified extensive use of packing, virtualization-based obfuscation, runtime assembly loading, and anti-analysis protections designed to conceal functionality and disrupt reverse engineering efforts.

The executable operates as a self-contained malware loader that embeds multiple compressed .NET assemblies directly within the binary using the Costura framework. Rather than writing additional files to disk, the malware dynamically loads required dependencies into memory during execution through reflection-based assembly loading techniques. Embedded libraries identified during analysis include components related to HTTP communication, SMTP functionality, JSON serialization, cryptographic operations, and graphical user interface elements.

Analysis of extracted strings and metadata revealed functionality consistent with:

- HTTP- and SMTP-based communication
- encrypted or serialized data handling
- asynchronous networking operations
- registry interaction
- process injection capabilities
- system and environment enumeration
- temporary file usage
- runtime resource loading
- anti-analysis behavior

The malware contains references to networking and communication libraries such as:

- [System.Net](#).Http
- HttpClient
- [Rebex.Net](#).Smtp
- Rebex.Mail
- Newtonsoft.Json

These components are commonly associated with applications that communicate with remote infrastructure for data transmission, updates, or remote communication workflows, potentially for command-and-control communication, credential exfiltration, payload delivery, or malware updates.

Multiple indicators strongly support the presence of advanced defense evasion mechanisms. The executable contains malformed or intentionally corrupted Intermediate Language (IL) code that caused decompilation failures and exceptions within dnSpy and ILSpy. Additional protections identified through Detect It Easy (DIE) and disassembly analysis include:

- virtualization-based obfuscation (XVM runtime)
- anti-debugging protections
- anti-ILDASM protections
- modified entry point behavior
- high-entropy packed sections
- dynamic API resolution
- runtime assembly unpacking

Disassembly performed in Cutter produced invalid instructions, incoherent control flow, and unrealistic memory references, strongly suggesting that portions of the executable are virtualized or encoded rather than directly executable native instructions.

Dynamic analysis produced minimal observable behavior within the FLARE VM analysis environment. The malware terminated shortly after execution without generating visible HTTP, DNS, or SMTP traffic. No meaningful process, registry, or filesystem activity attributable to the sample was observed. This behavior strongly suggests the presence of sandbox or virtual machine detection mechanisms designed to suppress execution during analysis.

Despite the limitations imposed by heavy obfuscation and virtualization protections, the combined static indicators strongly support the conclusion that the sample was designed to operate as a stealth-oriented malware loader capable of protected in-memory execution, remote communication, and potential credential or system data theft.



Malware Architecture Overview

AgentTesla consists of the following components:

File Name	SHA256 Hash
agenttesla.exe	2caae88b894455be6fd124fb2bff81452c7f5249d3afda7daa5567740300b88a

The analyzed sample is a heavily obfuscated .NET malware loader consistent with Agent Tesla-style information-stealing malware. Static analysis revealed that the executable operates as a self-contained packed payload that dynamically loads embedded dependencies directly from memory using the Costura framework.

The malware employs multiple layers of defense evasion, including:

- virtualization-based obfuscation (XVM runtime)
- malformed Intermediate Language (IL)
- anti-decompilation protections
- high-entropy packed sections
- runtime assembly loading
- dynamic API resolution

Unlike traditional malware that drops multiple files to disk, this sample embeds its required components directly inside the executable as compressed resources.

These components are decompressed and loaded into memory at runtime using reflection-based assembly loading techniques.

Analysis identified functionality associated with:

- HTTP and SMTP communications
- JSON serialization
- encrypted data handling
- possible process injection functionality
- registry interaction
- asynchronous networking
- environment awareness



- anti-analysis behavior

The malware appears designed to operate primarily in memory while minimizing filesystem artifacts and resisting both static and dynamic analysis.

Malware Execution Flow

Stage	Description	Indicators Identified
1. Initial Execution	Malware begins execution and initializes runtime environment.	PE64 .NET executable
2. Environment Validation	Checks execution environment and may verify privileges or detect analysis tools.	Anti-debugging behavior, NtTerminateProcess usage
3. Runtime Initialization	Loads embedded assemblies directly into memory using runtime loaders.	Costura embedded resources
4. Obfuscation Layer Activation	Virtualized/obfuscated runtime initializes to conceal actual logic.	XVM Runtime, malformed IL
5. Configuration Processing	Internal configuration or payload data is decoded/decrypted at runtime.	FromBase64String, CryptoStream, UTF8



6. System Enumeration	Contains indicators associated with system and environment information gathering..	RegistryKey, get_RegistryPath
7. Network Initialization	Initializes networking-related components associated with outbound communication functionality.	HttpClient, PostAsync, GetAsync
8. Data Serialization	Formats or processes data for communication.	JsonConvert, DeserializeObject, application/json
9. Malware Operations	Contains functionality associated with downloading, uploading, or outbound data transmission workflows.	UploadData, UploadString, DownloadFileWithProgressAsync
10. Cleanup / Evasion	Attempts to reduce artifacts or evade analysis.	CleanTempAsync, self-termination behavior

Operational Workflow Summary

The malware follows a staged execution model designed to delay exposure of malicious functionality until runtime. After execution, the sample initializes its protected runtime environment and dynamically loads embedded assemblies directly from memory. Obfuscation and virtualization protections are then activated to conceal internal logic and hinder analysis.

The malware appears to process encrypted or encoded configuration data before initializing networking functionality through HTTP- and SMTP-related libraries. Identified upload, download, and serialization routines suggest support for remote communication functionality, staged payload retrieval, and outbound data



transmission. Registry-related functionality additionally indicates possible persistence or host-specific configuration handling.

The combination of runtime assembly loading, encrypted configuration handling, asynchronous networking, and anti-analysis behavior strongly suggests the sample was designed for stealth-oriented execution within victim environments.

Key Findings

Category	Finding
<i>Malware Type</i>	<i>Obfuscated .NET malware loader</i>
<i>Likely Family</i>	<i>Agent Tesla-style stealer/loader</i>
<i>Architecture</i>	<i>64-bit PE (.NET)</i>
<i>Protection</i>	<i>XVM virtualization + anti-decompilation</i>
<i>Communication</i>	<i>HTTP/S + SMTP capability</i>
<i>Data Handling</i>	<i>JSON serialization + encrypted payload handling</i>
<i>Execution Style</i>	<i>In-memory assembly loading</i>
<i>Evasion</i>	<i>Anti-debugging and sandbox-aware behavior</i>
<i>Persistence Indicators</i>	<i>Registry interaction</i>
<i>Capabilities Identified</i>	<i>Upload, download, process injection, async networking</i>





Basic Static Analysis

Section Header Anomaly Summary

	pFile	Data	Description	Value
IEADER	00000188	08 68 74 1F	Name	ht= Fg
Program	0000018C	3D 7C 46 67		
ADERS	00000190	0005C9F4	Virtual Size	
	00000194	00002000	RVA	
E_HEADEI	00000198	0005CA00	Size of Raw Data	
TONAL_H	0000019C	00000400	Pointer to Raw Data	
ON_HEAD	000001A0	00000000	Pointer to Relocations	
ON_HEAD	000001A4	00000000	Pointer to Line Numbers	
ON_HEAD	000001A8	0000	Number of Relocations	
ON_HEAD	000001AA	0000	Number of Line Numbers	
ON_HEAD	000001AC	E0000040	Characteristics	
g		00000040	IMAGE_SCN_CNT_INITIALIZED_DATA	
%k		20000000	IMAGE_SCN_MEM_EXECUTE	
My		40000000	IMAGE_SCN_MEM_READ	
		80000000	IMAGE_SCN_MEM_WRITE	

During static inspection of the sample's PE section headers, one section exhibited a significant mismatch between its Virtual Size and Size of Raw Data, combined with a zeroed PointerToRawData. The Virtual Size (**0x0005C9F4**) indicates that the section expands to a large region in memory, while the Size of Raw Data (**0x0005CA00**) reflects only the aligned on-disk footprint. The absence of a valid PointerToRawData suggests that the section does not exist as a normal data block within the file and is instead populated dynamically at runtime.

This pattern is characteristic of packed or obfuscated malware, where the on-disk representation contains only an encrypted or compressed payload that is unpacked into memory during execution. The presence of executable, readable, and writable section characteristics further supports the likelihood of runtime unpacking or in-memory assembly loading. These traits align with common behaviors observed in Agent Tesla variants, which frequently rely on .NET obfuscation, custom loaders, and memory-only payload expansion to evade static detection.

Forged PE Header Timestamp

Agent Tesla Malware
April 2026
v1.0



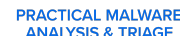
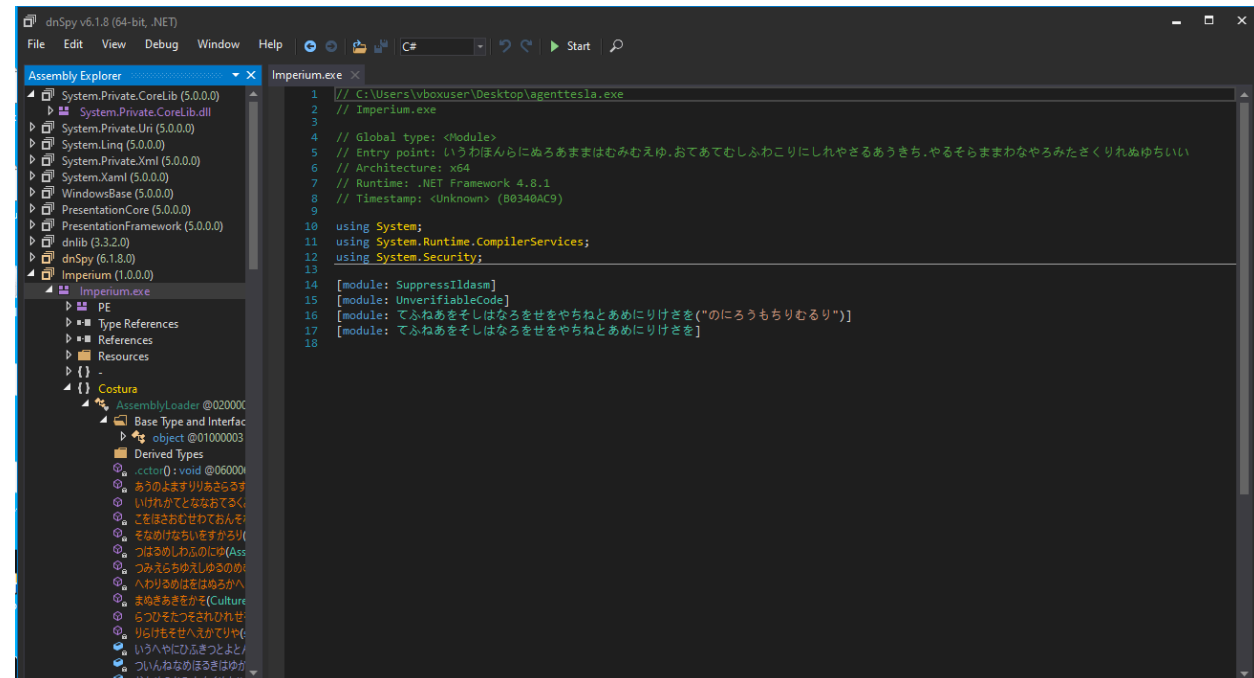
The PE header contains an invalid future timestamp (2063-09-05), which is not a plausible compile date for any legitimate software. This strongly indicates that the TimeDateStamp field has been deliberately forged. Malware authors commonly manipulate this value to evade signature-based detection, disrupt timeline reconstruction, and obscure the true origin or build chronology of the malware. The presence of a future timestamp is therefore a reliable static indicator of tampering and aligns with known anti-analysis techniques used by commodity malware families such as Agent Tesla.

property	value	value	value	value	value
section	section[0]	section[1]	section[2]	section[3]	section[4]
name	[ht= Fg	o!1m7?%kl	[7F9Wly	.text	.rsrc
section > sha256	A7308DD0663E8B4DC815D9...	32615351C3C4024697D3BFD...	BA827172A8FA6D18371736A...	93963440193A18B51BB4D1C...	DE29ACF667DC8853BD81A...
entropy	7.999	7.999	7.999	0.163	7.902
file > ratio (99.98%)	8.31 %	4.98 %	0.01 %	85.01 %	1.67 %
raw-address (begin)	0x00000400	0x0005CE00	0x00094600	0x00094800	0x00448A00
raw-address (end)	0x0005CE00	0x00094600	0x00094800	0x00448A00	0x0045B400
raw-size (4567040 bytes)	0x0005CA00 (379392 bytes)	0x00037800 (227328 bytes)	0x00000200 (512 bytes)	0x00384200 (3883520 bytes)	0x00012A00 (76288 bytes)
virtual-address (begin)	0x00002000	0x00060000	0x00098000	0x0009A000	0x00450000
virtual-address (end)	0x0005E9F4	0x00097620	0x00098008	0x0044E014	0x004628E1
virtual-size (4565265 bytes)	0x0005C9F4 (379380 bytes)	0x00037620 (226848 bytes)	0x00000008 (8 bytes)	0x00384014 (3883028 bytes)	0x000128E1 (76001 bytes)
characteristics	0xE0000040	0xE0000040	0xE0000040	0x60000020	0x40000040
write	x	x	x	-	-
execute	x	x	x	x	-
share	-	-	-	-	-
self-modifying	x	x	x	-	-
virtual	-	-	-	-	-
items					
directory > resource	-	-	-	-	0x00450000
directory > .NET	-	-	-	0x0009A000	-
manifest	-	-	-	-	0x0045A4A0
version	-	-	-	-	0x0045A184
base-of-code	-	-	-	0x0009A000	-

Agent Tesla Malware
April 2026
v1.0



PEStudio's static assessment immediately revealed multiple red flags consistent with a packed or obfuscated .NET malware sample. The file exhibited extremely high overall entropy (≈ 7.92), indicating that most of the content is compressed or encrypted. The import and library views were completely empty, an abnormal finding for any legitimate Windows or .NET application and a strong indicator that the executable relies on a custom loader that resolves APIs dynamically at runtime. The compiler timestamp was forged far into the future (2063), a common anti-forensic technique used to disrupt timeline analysis. The manifest requested elevated execution levels, including `requireAdministrator`, suggesting the program attempts to run with high privileges. Combined with the presence of randomized section names, high-entropy data blocks, and a .NET module labeled "Imperium.exe," PEStudio's results strongly support the conclusion that this file is a packed loader designed to conceal its true functionality until runtime.

[illegible]

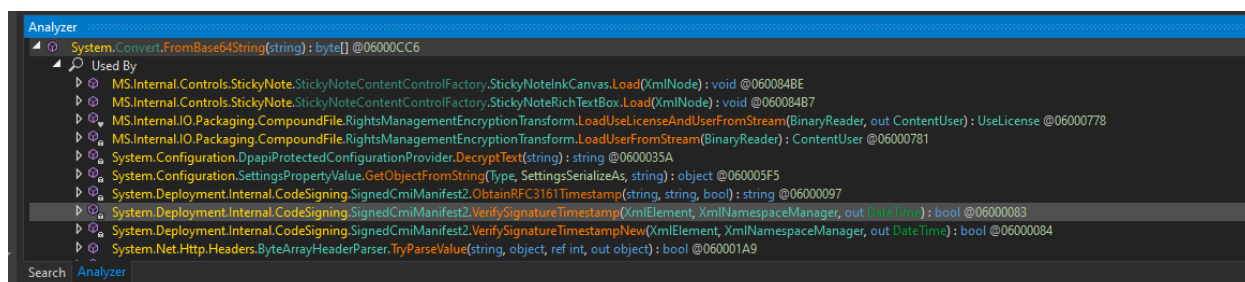
Agent Tesla Malware
April 2026
v1.0

Decompilation of the sample using dnSpy revealed the presence of the Costura framework, indicating that dependent assemblies are embedded within the executable and loaded dynamically at runtime. The **AssemblyLoader** class contains multiple obfuscated methods responsible for resolving and loading assemblies from the resource section using in-memory execution techniques.

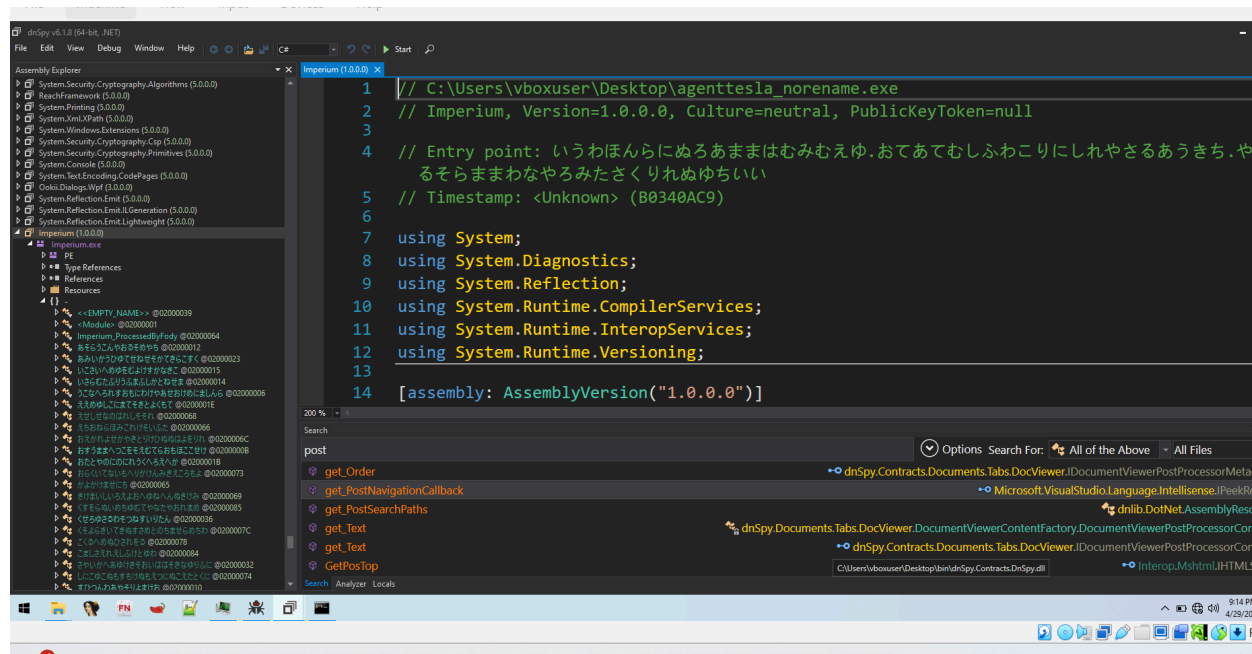
Several methods within this class could not be fully decompiled due to deliberate obfuscation, resulting in decompiler exceptions such as **OverflowException** and **ArgumentOutOfRangeException**. This behavior suggests the use of anti-analysis techniques designed to disrupt static reverse engineering efforts.

The loader appears to retrieve embedded resources as streams, convert them into byte arrays, and load them into memory using reflection (**Assembly.Load**). This approach allows the malware to execute its payload without writing additional files to disk, thereby evading traditional detection mechanisms.

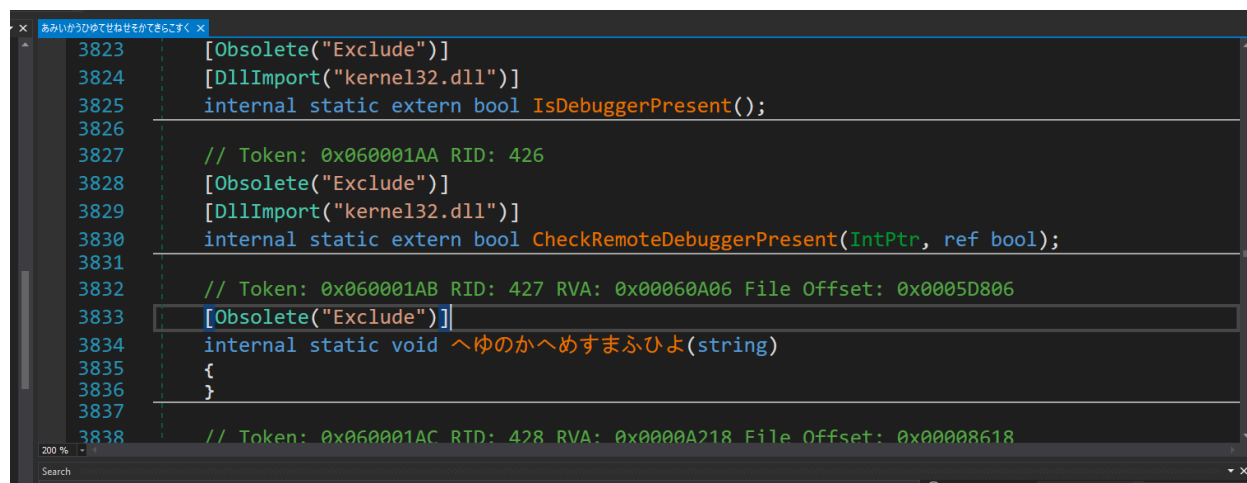
Combined with previously observed high entropy and abnormal section characteristics, these findings confirm that the sample operates as a packed .NET loader that conceals its primary functionality until runtime.



Analysis of Base64 decoding functions revealed no direct usage of standard .NET methods such as **Convert.FromBase64String** within the malware's own code. This suggests that the sample either employs custom encoding/decoding routines or retrieves its configuration dynamically at runtime. Multiple keyword searches, including "http", "POST", and "cmd", failed to identify meaningful indicators within the decompiled code. This strongly suggests that the malware employs string obfuscation or runtime decryption techniques to conceal its behavior.



The analyzed executable is named **agenttesla.exe**; however, the internal assembly metadata identifies the program as “Imperium.” This discrepancy suggests that the filename may have been altered to mimic a known malware family or mislead analysis, which is a common tactic used by malware authors.



Agent Tesla Malware
April 2026
v1.0



Advanced Static Analysis- Cutter

```
File Edit View Windows Debug Tools Help
Type flag name or address here

Functions
Name
entry0
fcn.00489f48
fcn.0049ce2c
fcn.004a759e
fcn.004a761e
fcn.004a8d5e
fcn.004aa62b
fcn.004aae37
fcn.004b1a47
fcn.004b52ba
fcn.0068e695
fcn.006dfe73
fcn.006fcf7f
fcn.007ddda3

-- section .text:
entry0(int64_t arg2);
; arg int64_t arg2 @ rdx
0x0049a000 add byte [rax], al ; [03] -r-x section size 3891200 named .text
0x0049a003 add byte [rdx], al ; arg2
0x0049a005 add byte [0x0a04b00b], al
0x0049a00b add ah, bh
0x0049a00d invalid
0x0049a00e invalid
0x0049a00f add byte [rax], al
0x0049a011 add byte [rax], al
0x0049a013 add byte [rdx + 0xc060005], ah
0x0049a019 and edx, dword [rcx]
0x0049a01b add byte [rax], cl
0x0049a01d mov ebp, 0x33 ; '3' ; 51
0x0049a022 add byte [rax], al
0x0049a024 add byte [rax], al
0x0049a026 add byte [rax], al
0x0049a028 add byte [rax], al
0x0049a02a add byte [rax], al
0x0049a02c add byte [rax], al
0x0049a02e add byte [rax], al
0x0049a030 add byte [rax], al
0x0049a032 add byte [rax], al
0x0049a034 add byte [rax], al
0x0049a036 add byte [rax], al
0x0049a038 add byte [rax], al
0x0049a03a add byte [rax], al
0x0049a03c add byte [rax], al
0x0049a03e add byte [rax], al
0x0049a040 add byte [rax], al
0x0049a042 add byte [rax], al
0x0049a044 add byte [rax], al
0x0049a046 add byte [rax], al
0x0049a048 invalid
0x0049a049 sub byte [rax], cl
0x0049a04b or eax, 0x80170600
0x0049a050 sbb ecx, dword [rdx]
0x0049a052 add byte [rax + rbp], al
0x0049a055 invalid
0x0049a056 or eax, 0x80160600
0x0049a05b sbb ecx, dword [rdx]
```

Disassembly of the sample using Cutter revealed largely non-meaningful instructions, including repeated `add byte [rax], al` operations and invalid opcodes. This indicates that the binary is likely packed or obfuscated, preventing direct analysis of its true functionality at the assembly level. The presence of such patterns suggests the use of a loader or packing mechanism, which defers execution of the actual malicious payload until runtime.



Cutter: Disassembly

```
fcn.006fcf7f();
0x006fcf7f      outsb  dx, byte [rsi]
0x006fcf81      xchg  esi, eax
0x006fcf82      cmpsd dword [rsi], dword ptr [rdi]
0x006fcf83      mov   cl, 0x49 ; 'I' ; 73
0x006fcf85      invalid
0x006fcf86      invalid
0x006fcf87      invalid
0x006fcf88      invalid
0x006fcf89      invalid
0x006fcf8a      pushfq
0x006fcf8c      fsubr st(5), st(0)
0x006fcf8e      mov   cl, 0xd8 ; 216
0x006fcf90      xor   al, 0x78 ; 120
0x006fcf93      add   al, 0x57 ; 87
0x006fcf95      adc   edx, edx
0x006fcf97      insd  dword [rdi], dx
0x006fcf98      invalid
0x006fcf99      invalid
0x006fcf9a      xchg  ebx, eax
0x006fcf9b      xor   dl, bl
0x006fcf9d      xor   esp, ebp
0x006fcf9f      ja    0x6fcf9e
0x006fcfa1      je    0x6fcf23
0x006fcfa3      mov   ebx, 0x1cfa6d9f
0x006fcfa8      invalid
0x006fcfa9      leave
0x006fcfaa      adc   byte [rsi], bl
0x006fcfac      jg    0x6fcffb
0x006fcfae      mov   bl, 0xe0 ; 224
0x006cfcb0      wait
0x006cfcb1      or    byte ss:[rax - 0x144b25f], bl
0x006cfcb8      invalid
0x006cfcb9      test  eax, 0xa0a06663
0x006cfcbf      jno   0x6fcf8e
0x006fcfc1      ret   0x98e6
0x006fcfc4      test  dword [rax], ebp
0x006fcfc6      out   0xc0, eax
0x006fcfc8      and   bl, byte [0xffffffff89daa411]
0x006fcfce      mov   es, word [rdx]
0x006fcfd0      mov   dl, 0xab ; 171
0x006fcfd2      pop   rbp
```

hboard	Strings	Imports	Search	Disassembly	Graph(fcn.006fcf7f)	Hexdump	Decompiler (entry0)
--------	---------	---------	--------	-------------	---------------------	---------	---------------------



Analysis of multiple functions revealed extensive use of invalid opcodes, rare instructions (e.g., **outsb**, **insd**, **fsubr**), and incoherent control flow. Memory references frequently pointed to unrealistic addresses, and instruction sequences lacked any meaningful logical structure. These characteristics strongly indicate that the binary employs advanced obfuscation techniques, likely virtualization-based, where actual program logic is encoded and interpreted at runtime rather than directly executed.

The disassembly output does not represent legitimate native instructions, but rather encoded data misinterpreted as assembly. This behavior is consistent with virtual machine (VM)-based obfuscation frameworks, such as XVM, which execute protected code through a custom runtime environment. As a result, traditional static disassembly is ineffective in revealing the true program logic.

```
Functions
Name
Import: false
Nargs: 0x1
Nbbs: 0x1
Nlocals: 0x0
Call type: ms
Edges: 0
StackFrame: 0
Comment: [03] -r-x section size 3891200 named .text
fcfn.00489f48
Offset: 0x00489f48
Size: 0x2d
Import: false
Nargs: 0x2
Nbbs: 0x3
Nlocals: 0x0
Call type: ms
Edges: 2
StackFrame: 0
Comment:
fcfn.0049ce2c
fcfn.004a759e
fcfn.004a761e
fcfn.004a8d5e
fcfn.004aa62b
fcfn.004aae37
fcfn.004b1a47
fcfn.004b52ba
fcfn.0068e695
fcfn.006dfe73
fcfn.006fc77f
fcfn.007dda3

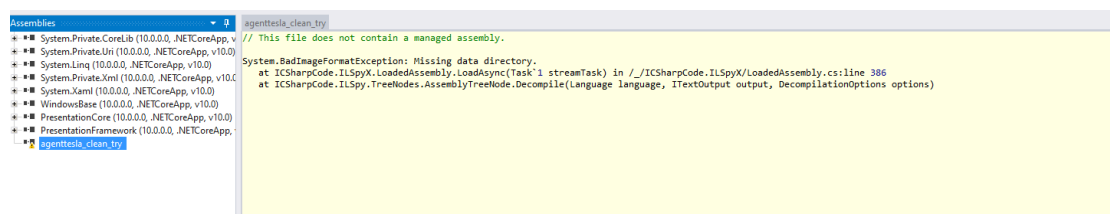
fcfn.0049ce2c();
0x0049ce2c  mov     dword [data.0059cf35], eax ; 0x59cf35
0x0049ce32  movsb   byte [rdi], byte ptr [rsi]
0x0049ce33  add     eax, 0
0x0049ce38  add     byte [rax], al
0x0049ce3a  xor     dword [rax], eax
0x0049ce3c  wait
0x0049ce3d  add     al, 0x93 ; 147
0x0049ce3f  add     eax, 0x100103
0x0049ce44  invalid
0x0049ce45  invalid
0x0049ce46  add     byte [rax], al
0x0049ce48  add     byte [rax], al
0x0049ce4a  add     byte [rax], al
0x0049ce4c  xor     dword [rax], eax
0x0049ce4e  movabs  al, byte [0xfb00100180059d04]
0x0049ce57  rol     dword [rdx], 0
0x0049ce5a  xchg    edx, eax
0x0049ce5b  fld     dword [rax]
0x0049ce5d  add     byte [0xfffffffffb14e7663], cl
0x0049ce63  add     eax, 0x100001
0x0049ce68  jnp     0x49ced4
0x0049ce6a  add     eax, dword [rax]
0x0049ce6c  invalid
0x0049ce6d  invalid
0x0049ce6e  add     eax, dword [rax]
0x0049ce70  or      eax, 0xce04b600
0x0049ce75  add     eax, 0x100103
0x0049ce7a  jno     0x49ce7e
0x0049ce7c  add     byte [rax], al
0x0049ce7e  add     byte [rax], al
0x0049ce80  add     byte [rax], al
0x0049ce82  xor     dword [rax], eax
0x0049ce84  enter   0xf04, 6 ; 3844
0x0049ce88  add     dword [rcx], 0xcd260010
0x0049ce8e  add     eax, dword [rax]
0x0049ce90  ret     0x186
0x0049ce93  add     byte [0x134e9a99], cl
0x0049ce99  invalid
0x0049ce9a  add     byte [rax], al
0x0049ce9c  adc     byte [rax], al
0x0049ce9e  rcr     byte [rax - 0x32f8fffe], 1
```



Examination of individual functions revealed a lack of meaningful program logic, with numerous invalid instructions and repetitive low-level operations (e.g., `add byte [rax], al`). Memory references pointed to unrealistic addresses, and control flow appeared inconsistent and non-linear. These characteristics strongly indicate the presence of obfuscation or virtualization, where executable code is intentionally distorted to prevent static analysis.

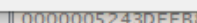
The analyzed functions appear to be artificially constructed stubs rather than genuine logic, likely generated by an obfuscation or virtualization framework. The prevalence of invalid opcodes and nonsensical instruction sequences suggests that the disassembler is interpreting encoded or virtualized instructions as native assembly.

Runtime Unpacking & Memory Analysis



Due to the extensive obfuscation and virtualization protections present within the sample, traditional static decompilation methods were insufficient to fully recover the malware's runtime logic and embedded functionality. Multiple runtime memory extraction and process dumping techniques were therefore employed in an attempt to reconstruct the protected .NET assemblies and recover additional runtime artifacts.

Initial analysis using dnSpy and ILSpy resulted in repeated decompilation failures and CLR metadata exceptions, suggesting the malware intentionally employed malformed Intermediate Language (IL), runtime virtualization, or protected metadata structures designed to disrupt reverse engineering workflows.



- process memory dumping
- raw memory extraction
- rebuild-based PE reconstruction
- runtime string extraction
- memory artifact recovery

- process memory dumping
- raw memory extraction
- rebuild-based PE reconstruction
- runtime string extraction
- memory artifact recovery

Agent Tesla Malware
April 2026
v1.0



Although a fully reconstructed executable could not be recovered, runtime memory inspection successfully identified numerous embedded libraries, networking indicators, cryptographic routines, serialization components, and anti-analysis artifacts not directly visible through static decompilation alone.

Recovered runtime artifacts included:

- HTTP and SMTP communication indicators
- Base64 decoding functionality
- cryptographic stream handling
- runtime assembly loading components
- registry interaction indicators
- XVM virtualization-related artifacts
- process injection references
- asynchronous networking methods

The inability to fully reconstruct the executable despite multiple dump and rebuild attempts strongly suggests the malware employs advanced runtime protections designed to hinder memory reconstruction, debugger-assisted analysis, and automated tooling.



Embedded Resource & String Analysis

application/json				
POST				
Win32_BaseBoard	00000000	00000000	Machine	IMAGE_FILE_MACHINE_AMD64
SerialNumber	00000000	00000000	Machine	IMAGE_FILE_MACHINE_AMD64
cmd.exe	00000000	00000000	Number of Sections	
System	00000000	00000000	Time Date Stamp	2023/09/05 Wed 06:52:49 UTC
0Harmony	00000000	00000000	Pointer to Symbol Table	
noKey	00000000	00000000	Number of Symbols	
noValueButYesKey	00000000	00000000	Size of Optional Header	
DLL name cannot be null or empty	00000000	00000000	Characteristics	
func	00000000	00000000	Image File Characteristics	IMAGE_FILE_EXECUTABLE_IMAGE
Function name cannot be null or empty	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
.dll	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
Failed to load library '{0}'. Win32 Error: {1} (0x{2:X})				
Failed to find function '{0}' in '{1}'. Tried: {2}, {3}A, {4}W. Win32 Error: {5} (0x{6:X})				
.DLL	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
pm p	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
sVEuz	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
ujSo	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
nyYF	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
http://	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
.com	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
temp.txt	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
.txt	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
.compressed	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
costura	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
costura.costura.dll.compressed	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
costura.costura.pdb.compressed	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
guna.ui2	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
costura.guna.ui2.dll.compressed	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
newtonsoft.json	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
costura.newtonsoft.json.dll.compressed	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
sharpdx.directinput	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
costura.sharpdx.directinput.dll.compressed	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
costura.sharpdx.directinput.pdb.compressed	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
sharpdx	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
costura.sharpdx.dll.compressed	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
costura.sharpdx.pdb.compressed	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
.txt	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
.txt	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
http://	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
.com	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE
.txt	00000000	00000000	Image File Characteristics	IMAGE_FILE_LARGE_ADDRESS_AWARE

String extraction from the sample revealed numerous indicators consistent with a heavily packed and obfuscated .NET malware payload. Multiple embedded resource entries were identified using the naming convention:

- **costura.*.dll.compressed**

including:

- **costura.costura.dll.compressed**
- **costura.newtonsoft.json.dll.compressed**
- **costura.guna.ui2.dll.compressed**
- **costura.sharpdx.dll.compressed**

These artifacts strongly indicate the use of the Costura/Fody framework, which embeds dependent .NET assemblies directly into the executable as compressed



resources. While Costura itself is a legitimate packaging framework, its use within malware is common because it allows malicious components to be loaded directly from memory without writing additional files to disk.

Additional runtime strings identified references to several embedded .NET libraries, including:

- newtonsoft.json
- guna.ui2
- sharpdx

These libraries suggest functionality associated with JSON serialization, graphical interface rendering, and runtime interaction capabilities.

Numerous networking- and runtime-related strings were also identified, including:

- http://
- .com
- application/json
- UploadData
- PostAsync

These indicators suggest support for outbound HTTP communication and structured data transmission.

Additional error-handling strings such as:

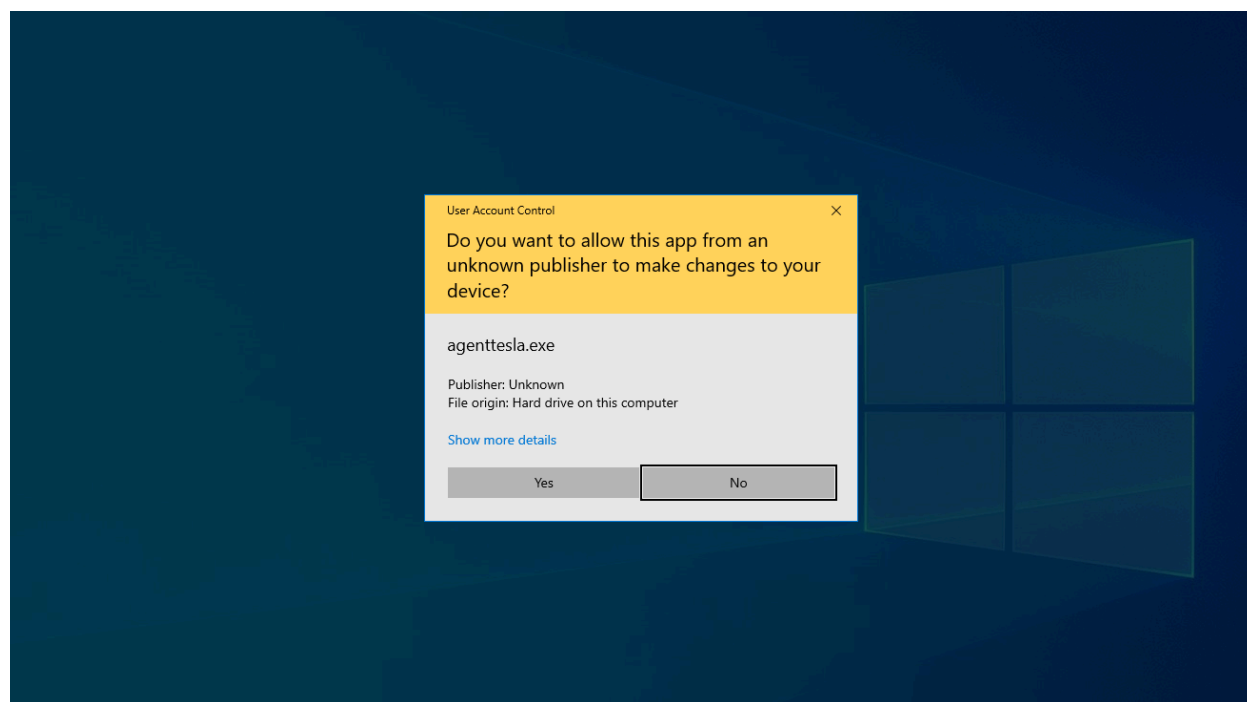
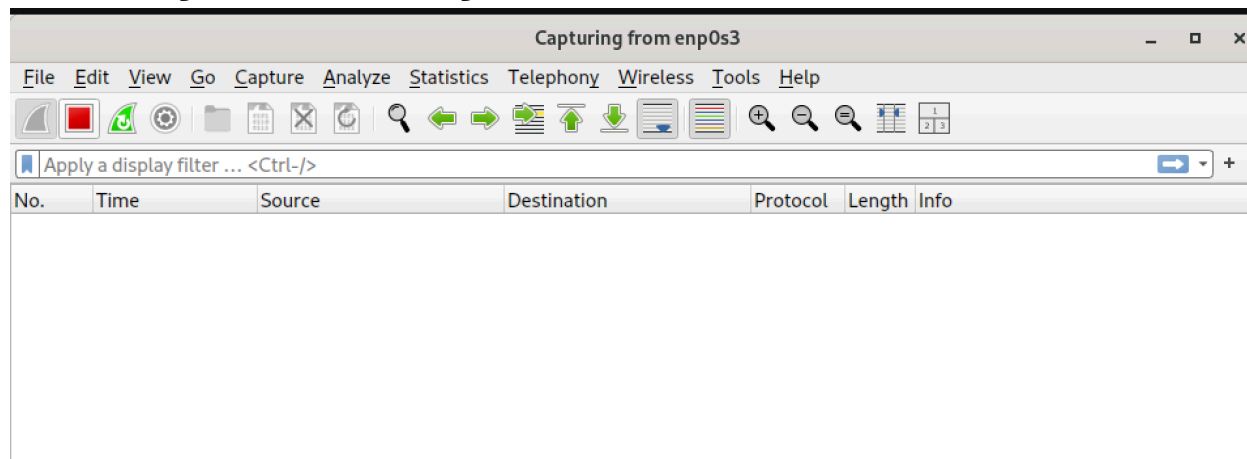
- DLL name cannot be null or empty
- Failed to load library
- Failed to find function

suggest the malware dynamically resolves APIs or loads assemblies at runtime, a technique commonly associated with obfuscation and anti-analysis behavior.

Collectively, these artifacts reinforce the assessment that the sample operates as a packed and obfuscated .NET malware loader utilizing embedded resources, runtime assembly loading, and concealed execution logic to hinder reverse engineering efforts.



Basic Dynamic Analysis





The sample was executed within a controlled FLARE VM environment while monitoring filesystem, registry, process, and network activity using Process Monitor and Wireshark.

Upon execution, the malware terminated shortly after launch without generating significant observable behavior. No persistent child processes, file modifications, registry changes, DNS requests, HTTP traffic, or SMTP communications were detected during execution.

Debugger-assisted analysis identified repeated invocation of **NtTerminateProcess**, strongly suggesting deliberate self-termination behavior consistent with anti-analysis or virtualization-detection mechanisms. This behavior likely prevented the malware from fully executing within the monitored environment.

Although live outbound communication was not successfully observed, prior static analysis and runtime string extraction identified multiple indicators associated with:

- HTTP communication
- SMTP functionality
- asynchronous networking
- encrypted data handling
- staged payload retrieval

The absence of runtime activity, combined with extensive static indicators of obfuscation and runtime assembly loading, strongly suggests the malware employs sandbox-aware execution controls intended to suppress malicious functionality during analysis.

Advanced Dynamic Analysis

9.03.5. rtasking.exe	6788 RegOpenKey	HKEY\Software\Classes	SUCCESS	Query: HandleTags, HandleTags: 0x0
9.03.5. rtasking.exe	6788 RegOpenKey	HKEY\Software\Classes	SUCCESS	Query: HandleTags, HandleTags: 0x0
9.03.5. rtasking.exe	6788 RegOpenKey	HKEY\Software\Classes\CLSID\{56AD4C3D-8908-4F85-BFF1-7940C29B3BCF}\Instance	NAME NOT FOUND	Desired Access: Read
9.03.5. rtasking.exe	6788 RegOpenKey	HKEY\CLSID\{56AD4C3D-8908-4F85-BFF1-7940C29B3BCF}\Instance	NAME NOT FOUND	Desired Access: Read
9.03.5. rtasking.exe	2548 DeviceIoControl	\Device\NPF	NO MORE MATC...	Control: 0x140364 (DeviceIoControl) Function: 233 Method: 0
9.03.5. rtasking.exe	3496 Thread Create		SUCCESS	Thread ID: 6396
9.03.5. rtasking.exe	3496 Thread Create		SUCCESS	Thread ID: 7472
9.03.5. rtasking.exe	2600 RegOpenKey	HKEY\Software\Microsoft\Office\ClickToRun	SUCCESS	Query: HandleTags, HandleTags: 0x0
9.03.5. rtasking.exe	2600 RegOpenKey	HKEY\Software\Microsoft\Office\ClickToRun\ExecutingScenario	SUCCESS	Desired Access: Read
9.03.5. rtasking.exe	2600 RegOpenKey	HKEY\Software\Microsoft\Office\ClickToRun	NAME NOT FOUND	Length: 12
9.03.5. rtasking.exe	2600 RegOpenKey	HKEY\Software\Microsoft\Office\ClickToRun\Configuration	SUCCESS	Query: HandleTags, HandleTags: 0x0
9.03.5. rtasking.exe	2600 RegOpenKey	HKEY\Software\Microsoft\Office\ClickToRun\Configuration\TimeInterval	SUCCESS	Desired Access: Read
9.03.5. rtasking.exe	2600 RegOpenKey	HKEY\Software\Microsoft\Office\ClickToRun\Configuration	NAME NOT FOUND	Length: 12
9.03.5. rtasking.exe	628 CreateFile	C:\Windows\ServiceState\EventLog	SUCCESS	Desired Access: Read Data/List Directory, Synchronize, Disposition: Create, O...
9.03.5. rtasking.exe	628 CreateFile	C:\Windows\ServiceState\EventLog\Data	NAME COLLISION	Desired Access: Read Data/List Directory, Synchronize, Disposition: Create, O...
9.03.5. rtasking.exe	1112 CreateFile	C:\Windows\ServiceState\EventLog\Data\Install1.dat	SUCCESS	Desired Access: Generic Write, Disposition: OverwriteIf, Options: Synchronous I...
9.03.5. rtasking.exe	4384 FileSystemControl		SUCCESS	Control: FSCTL_QUERY_USN_JOURNAL
9.03.5. rtasking.exe	4384 FileSystemControl		SUCCESS	Control: FSCTL_READ_USN_JOURNAL
9.03.5. rtasking.exe	4384 FileSystemControl		SUCCESS	Control: FSCTL_READ_USN_JOURNAL
9.03.5. rtasking.exe	1112 WriteFile	C:\Windows\ServiceState\EventLog\Data\Install1.dat	SUCCESS	Offset: 0, Length: 2,048, Priority: Normal
9.03.5. rtasking.exe	1112 WriteFile	C:\Windows\ServiceState\EventLog\Data\Install1.dat	SUCCESS	Control: FSCTL_QUERY_USN_JOURNAL
9.03.5. rtasking.exe	1112 CloseFile		SUCCESS	
9.04.0. rtasking.exe	6788 RegOpenKey	HKEY\SYSTEM\CurrentControlSet\Control\WMI\Restrictions	REPARSE	Desired Access: Read
9.04.0. rtasking.exe	6788 RegOpenKey	HKEY\SYSTEM\CurrentControlSet\Control\WMI\Restrictions	NAME NOT FOUND	Desired Access: Read
9.04.0. rtasking.exe	6788 RegOpenKey	HKEY\SYSTEM\CurrentControlSet\Control\WMI\Restrictions	REPARSE	Desired Access: Read
9.04.0. rtasking.exe	6788 RegOpenKey	HKEY\SYSTEM\CurrentControlSet\Control\WMI\Restrictions	NAME NOT FOUND	Desired Access: Read

Process Monitor Analysis

Agent Tesla Malware
April 2026
v1.0



Process Monitor was used to capture file system, registry, and process activity during execution of the sample. Despite multiple execution attempts and filtering for relevant processes, no activity attributable to the malware was observed.

Observed activity consisted solely of legitimate Windows processes such as `svchost.exe`, `services.exe`, and `OfficeClickToRun.exe`. No evidence of file creation, registry modification, or process execution associated with the sample was identified.

This absence of observable behavior strongly suggests that the malware either failed to execute or terminated immediately after performing environment checks. This behavior is consistent with sandbox or virtual environment detection techniques designed to evade analysis

String Analysis

Static and runtime string extraction revealed numerous indicators associated with networking functionality, runtime assembly loading, cryptographic handling, system interaction, and protected execution behavior. The recovered strings strongly support the assessment that the sample is a heavily obfuscated .NET malware loader designed to conceal functionality until runtime execution.

Embedded Resources & Runtime Components

String extraction identified multiple embedded resource entries using the naming convention:

- `costura.*.dll.compressed`

including:

- `costura.costura.dll.compressed`
- `costura.newtonsoft.json.dll.compressed`
- `costura.guna.ui2.dll.compressed`
- `costura.sharpx.dll.compressed`

These artifacts strongly indicate the use of the Costura/Fody framework, which embeds dependent .NET assemblies directly within the executable as compressed resources. This technique enables the malware to dynamically load assemblies from



memory without dropping additional files to disk, reducing filesystem artifacts and complicating static analysis.

Additional embedded library indicators included:

- newtonsoft.json
- guna.ui2
- sharpdx
- XVM.Runtime

These libraries suggest functionality related to JSON serialization, graphical interface rendering, runtime interaction, and virtualization-based obfuscation.



```
openmeau
get_CurrentThread
GetCurrentThread
Thread
Thread
Thread
Thread
Thread
Thread
add_Load
OnLoad
get_Red
Interlocked
set_Enabled
add_FormClosed
get_IsDisposed
costura.costura.pdb.compressed
costura.sharpx.directinput.pdb.compressed
costura.sharpx.pdb.compressed
costura.guna.ui2.dll.compressed
costura.costura.dll.compressed
costura.newtonsoft.json.dll.compressed
costura.sharpx.directinput.dll.compressed
costura.sharpx.dll.compressed
get_IsHandleCreated
set_Animated
AwaitUnsafeOnCompleted
get_IsCompleted
get_HasExited
get_IsCancellationRequested
System.Collections.Specialized
Synchronized
Void
NewGuid
hwnd
<Name>i__Field
<Url>i__Field
<SessionId>k__BackingField
GetField
Stsfld
get_Hand
Append
RegistryValueKind
set_IsBackground
<
```

Figure 1. Embedded Costura Resources and Runtime Components

The extracted runtime strings reveal multiple embedded compressed DLL resources and runtime libraries associated with obfuscated .NET malware loaders.

Networking & Communication Indicators

Agent Tesla Malware
April 2026
v1.0



Multiple networking-related strings and asynchronous communication methods were identified during runtime analysis, including:

- `HttpClient`
- `HttpResponseMessage`
- `GetAsync`
- `PostAsync`
- `UploadData`
- `UploadString`
- `application/json`
- `smtp-mail.outlook.com`
- `Rebex.Net.Smtp`
- `Rebex.Mail`

These artifacts suggest the presence of networking-related functionality associated with outbound HTTP- and SMTP-based communication and structured data transmission workflows. While direct outbound communication was not observed during execution, the recovered strings and libraries are consistent with components commonly used for remote communication and data transfer operations.

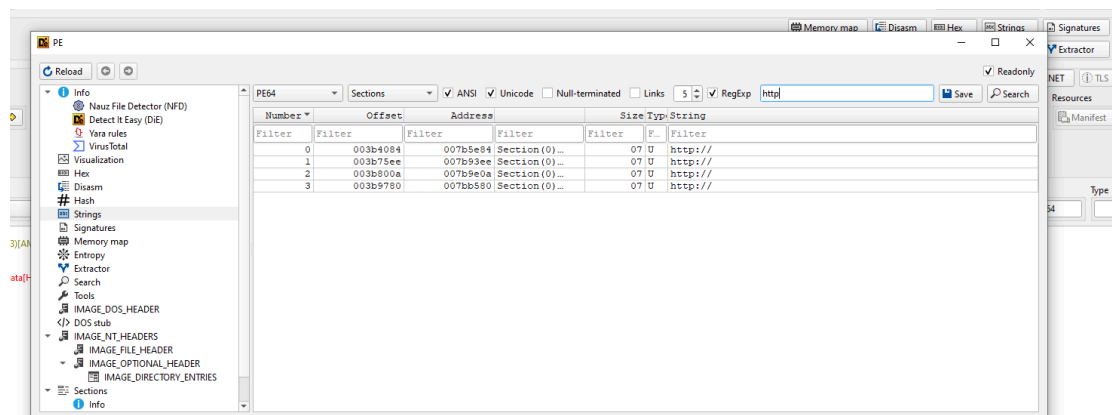
The presence of asynchronous networking methods such as `PostAsync` and `DownloadFileWithProgressAsync` further indicates that the malware is capable of maintaining network communication while performing concurrent runtime operations.



```
Failed to terminate process with PID {0}.
application/json
POST
Win32_BaseBoard
SerialNumber
cmd.exe
System
0Harmony
noKey
noValueButYesKey
DLL name cannot be null or empty
func
Function name cannot be null or empty
.dll
Failed to load library '{0}'. Win32 Error: {1} (0x{2:X})
Failed to find function '{0}' in '{1}'. Tried: {2}, {3}A, {4}W. Win32 Error: {5} (0x{6:X})
.DLL
pm|p
sVEuz
ujSo
nyYF
http://
.com
temp.txt
.txt
.compressed
costura
costura.costura.dll.compressed
costura.costura.pdb.compressed
guna.ui2
costura.guna.ui2.dll.compressed
newtonsoft.json
costura.newtonsoft.json.dll.compressed
```

Figure 2. Networking and HTTP Communication Indicators

Runtime string analysis identified multiple networking-related methods associated with HTTP requests, asynchronous communication, and outbound data transmission functionality.



Cryptographic & Serialization Indicators

Several strings associated with cryptographic processing, encoding, and serialized data handling were recovered during analysis, including:

- `CryptoStream`
- `CryptoStreamMode`
- `FromBase64String`
- `JsonConvert`
- `DeserializeObject`
- `UTF8`
- `GetString`
- `System.Security.Cryptography.X509Certificates`

These artifacts suggest support for processing encrypted, encoded, or serialized configuration and communication data during runtime execution., encoded, or serialized configuration and communication data during runtime execution. The presence of cryptographic streams and Base64 decoding functionality strongly indicates efforts to conceal configuration values, payloads, or outbound traffic from static inspection.



Number	Offset	Address	Size	Type	String
0	0036344f	0076524f	0c	A	CryptoStream
1	0036348b	0076528b	10	A	CryptoStreamMode

Figure 3. Cryptographic and Serialization Indicators

Recovered runtime strings associated with cryptographic processing and serialized data handling suggest support for encrypted configuration storage and protected network communication.

Number	Offset	Address	Size	Type	String
0	00363b9e	0076599e	10	A	FromBase64String

Figure 4. Runtime Base64 decoding functionality associated with protected configuration handling.

System Interaction & Environment Awareness

String analysis additionally revealed indicators associated with system interaction, host profiling, and runtime environment awareness, including:

- [GetHostName](#)
- [get_UserName](#)
- [RegistryKey](#)



- `get_RegistryPath`
- `Win32_BaseBoard`
- `SerialNumber`
- `Local\\Temp`
- `temp.txt`
- `cmd.exe`

These artifacts indicate the malware is capable of interacting with the Windows operating system, gathering host-specific information, accessing registry structures, and utilizing temporary filesystem locations during execution.

Additional runtime strings such as:

- `GetProcessesByName`
- `GetEnvironmentVariable`
- `System.Diagnostics`

suggest process enumeration and environment-awareness functionality that may be used for sandbox detection, targeting, or anti-analysis behavior.

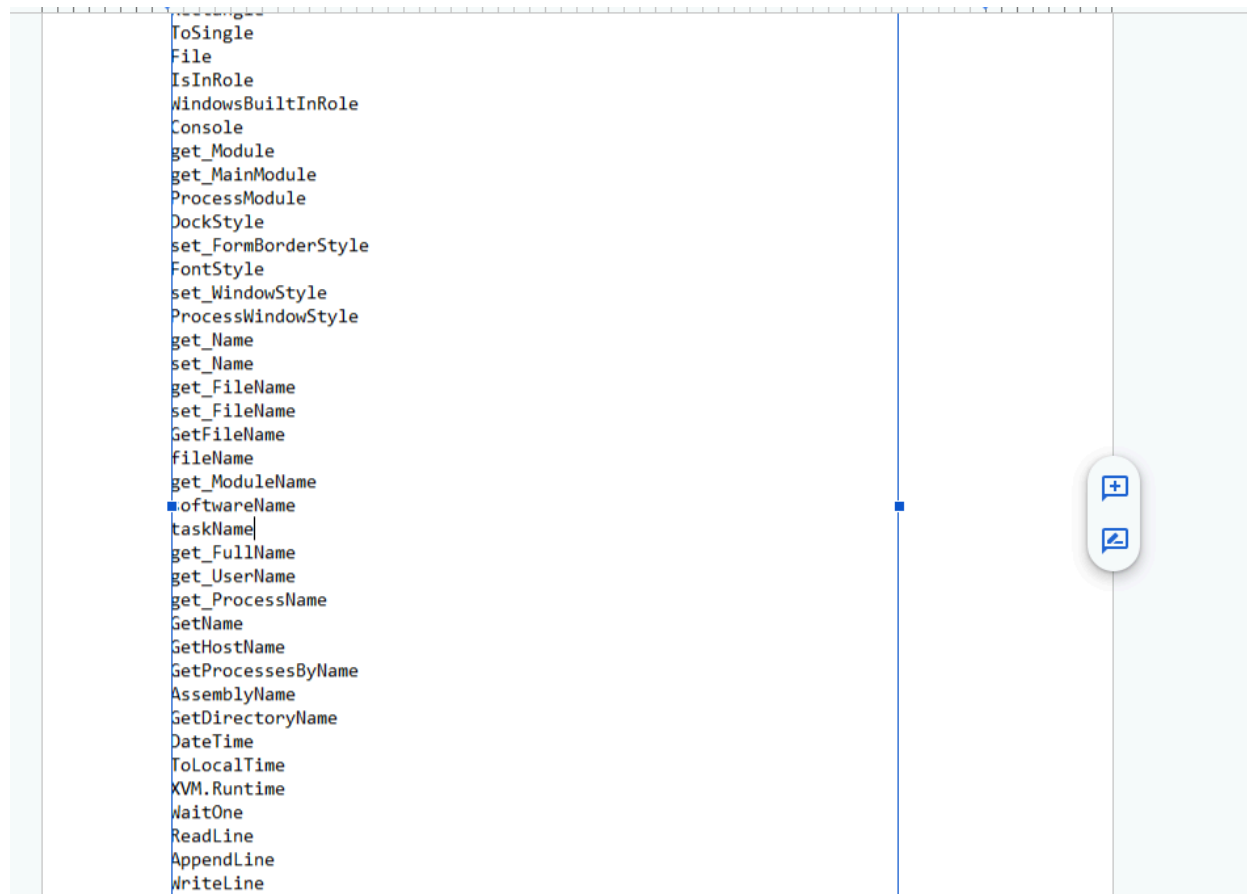


Figure 5. System Interaction and Environment Awareness Indicators

Recovered runtime strings indicate interaction with registry structures, temporary filesystem locations, host profiling functionality, and process enumeration behavior.

Injection & Evasion Indicators

Further analysis revealed multiple strings associated with execution control, injection functionality, and anti-analysis behavior, including:

- **ProcessInjection**
- **DriverInject**
- **SuccessInject**
- **InitAsync**



- ListenLoop
- CleanTempAsync
- NtTerminateProcess

These artifacts suggest the presence of functionality associated with process injection, staged execution workflows, runtime cleanup operations, and environment-aware execution controls. The presence of injection-related functionality, combined with repeated self-termination behavior observed during debugging, supports the assessment that the malware employs multiple defense-evasion and anti-analysis mechanisms.

Additional dynamic loading error strings including:

- Failed to load library
- Failed to find function
- DLL name cannot be null or empty

suggest that the malware dynamically resolves APIs or reconstructs assemblies during runtime execution, further complicating static analysis and reverse engineering efforts.



```
<+inaIPatnCneat>>__4  
<IsUnderMaintenance>d__4  
<>u__4  
<InitAsync>d__25  
<SuccessInject>d__45  
<ButtonDirectX_Click>d__65  
get_hero_image_fill_size_1200x675  
<processName>5__5  
<lastUpdate>5__5  
<dll>5__5  
<GetAllCheatStatuses>d__5  
<DriverInject>d__36  
<ProcessInjeccion>d__46  
HMACSHA256  
<ButtondControl_Click>d__66  
<fullPath>5__6  
<stream>5__6  
<tempVigem>5__6  
<ValidateLicense>d__6  
<loginBtn_Click>d__27  
<ButtonViGEmBus_Click>d__67  
<tempPath>5__7  
<tempSdl>5__7  
<fileStream>5__7  
get_f2a671233a34e4e3bc389b15727440b7  
get_pngaaa_com_6147418  
get_icons8_red_de_batalla_48  
get_icons8_wifi_48  
get_icons8_steam_48  
get_icons8_xbox_48  
get_UTF8  
<d1>5__8  
<buffer>5__8  
<Status>d__29  
<BnetButton_Click>d__49  
<bytesRead>5__9  
<
```



```
Rectangle
ToSingle
File
IsInRole
WindowsBuiltInRole
Console
get_Module
get_MainModule
ProcessModule
DockStyle
set_FormBorderStyle
FontStyle
set_WindowStyle
ProcessWindowState
get_Name
set_Name
get_FileName
set_FileName
GetFileName
fileName
get_ModuleName
softwareName
taskName|
get_FullName
get_UserName
get_ProcessName
GetName
GetHostName
GetProcessesByName
AssemblyName
GetDirectoryName
DateTime
ToLocalTime
XVM.Runtime
WaitOne
ReadLine
AppendLine
WriteLine
Combine
IAsyncStateMachine
SetStateMachine
stateMachine
LocalMachine
<
```

Figure 6. Injection and Anti-Analysis Indicators

String extraction identified multiple indicators associated with process injection, runtime cleanup behavior, and anti-analysis functionality intended to hinder reverse engineering and monitored execution.

Methodology

Analysis was conducted within a controlled FLARE VM malware analysis environment using a combination of static analysis, debugger-assisted runtime analysis, behavioral monitoring, runtime memory inspection, and string extraction techniques.

Static analysis focused on PE structure inspection, entropy analysis, embedded resource identification, disassembly, and .NET metadata inspection. Dynamic analysis involved monitored execution of the sample while observing process, filesystem, registry, and network activity.

Runtime memory inspection and reconstruction attempts were performed to recover protected assemblies and runtime artifacts not directly accessible through standard decompilation methods.

Tools utilized during analysis included:

- dnSpy
- ILSpy
- Cutter
- PEStudio
- Detect It Easy (DIE)



- Process Monitor
- Wireshark
- FLOSS
- OllyDumpEx
- FLARE VM

Due to extensive obfuscation, virtualization protections, malformed Intermediate Language (IL), and anti-analysis behavior present within the sample, complete runtime reconstruction and full behavioral execution could not be conclusively observed.

Screenshot Provenance

All screenshots included within this report were generated during analysis performed within a controlled FLARE VM environment using the tools identified in the Methodology section. Screenshots represent analyst-observed output captured during static analysis, runtime inspection, debugger-assisted analysis, string extraction, memory reconstruction attempts, and behavioral monitoring workflows.

Observed Behavior vs Inferred Capability

The analysis identified multiple directly observed indicators associated with runtime assembly loading, virtualization-based obfuscation, malformed Intermediate Language (IL), anti-debugging behavior, and protected execution techniques. These behaviors were confirmed through static analysis, debugger-assisted inspection, runtime memory analysis, and extracted runtime artifacts.

Observed behaviors included:

- Costura-based runtime assembly loading
- malformed and protected .NET metadata
- XVM-related virtualization indicators



- anti-debugging and self-termination behavior
- high-entropy packed sections
- runtime string extraction artifacts
- embedded networking and serialization libraries

Additional functionality discussed throughout this report, including possible credential theft, staged payload handling, process injection, outbound communication, and data exfiltration behavior, is based on inferred capability derived from recovered runtime strings, embedded libraries, API references, and execution characteristics rather than directly observed malicious execution.

Due to extensive obfuscation, protected runtime behavior, and limited observable execution within the FLARE VM environment, several operational capabilities could not be conclusively validated during dynamic analysis.

Analysis Limitations

```
namespace てうちかきそつやひのとすね
{
    // Token: 0x0200003A RID: 58
    internal class つみふまないほをおされそりぬするわいとけわのろ
    {
        // Token: 0x060002C1 RID: 705 RVA: 0x00050A00 File Offset: 0x000032A0
        private Task<string> せそふぬしおよなきみたねりぬなお(Dictionary<string, string> えぬのはやくそのこさすえつけし)
        {
            /*
            An exception occurred when decompiling this method (060002C1)

            ICSharpCode.Decompiler.DecompilerException: Error decompiling System.Threading.Tasks.Task`1<System.String> てうちかきそつやひのとすね.つみふまないほをおされそりぬするわいと
            けわのろ.:せそふぬしおよなきみたねりぬなお(System.Collections.Generic.Dictionary`2<System.String,System.String>)

            ----> System.OverflowException: Arithmetic operation resulted in an overflow.
            at ICSharpCode.Decompiler.ILast.ILastBuilder.StackSlot.ModifyStack(StackSlot[] stack, Int32 popCount, Int32 pushCount, ByteCode pushDefinition) in D:\a
            \dnSpy\dnSpy\Extensions\ILSpy.Decompiler\ICSharpCode.Decompiler\ICSharpCode.Decompiler\ILAst\ILAstBuilder.cs:line 47
            at ICSharpCode.Decompiler.ILast.ILastBuilder.StackAnalysis(MethodDef methodDef) in D:\a\dnSpy\dnSpy\Extensions\ILSpy.Decompiler\ICSharpCode.Decompiler
            \ICSharpCode.Decompiler\ILAst\ILAstBuilder.cs:line 387
            at ICSharpCode.Decompiler.ILast.ILastBuilder.Build(MethodDef methodDef, Boolean optimize, DecompilerContext context) in D:\a\dnSpy\dnSpy\Extensions
            \ILSpy.Decompiler\ICSharpCode.Decompiler\ICSharpCode.Decompiler\ILAst\ILAstBuilder.cs:line 271
            at ICSharpCode.Decompiler.Ast.AstMethodBodyBuilder.CreateMethodBody(IEnumerable`1 parameters, MethodDebugInfoBuilder& builder) in D:\a\dnSpy\dnSpy
            \Extensions\ILSpy.Decompiler\ICSharpCode.Decompiler\ICSharpCode.Decompiler\Ast\AstMethodBodyBuilder.cs:line 112
            at ICSharpCode.Decompiler.Ast.AstMethodBodyBuilder.CreateMethodBody(MethodDef methodDef, DecompilerContext context, AutoPropertyProvider
            autoPropertyProvider, IEnumerable`1 parameters, Boolean valueParameterIsKeyword, StringBuilder sb, MethodDebugInfoBuilder& stmtsBuilder) in D:\a\dnSpy\dnSpy
            \Extensions\ILSpy.Decompiler\ICSharpCode.Decompiler\ICSharpCode.Decompiler\Ast\AstMethodBodyBuilder.cs:line 99
            --- End of inner exception stack trace ---
            
```

The analyzed sample employed multiple anti-analysis and protection mechanisms that significantly complicated reverse engineering efforts. Static and dynamic analysis identified evidence of virtualization-based obfuscation, malformed



Intermediate Language (IL), anti-debugging behavior, runtime assembly loading, and anti-decompilation protections designed to disrupt traditional analysis workflows.

Attempts to fully decompile the executable using standard .NET analysis tools resulted in repeated failures, including `DecompilerException` and `System.OverflowException` errors, indicating the presence of intentionally malformed or protected CLR metadata structures. Runtime memory reconstruction and assembly dumping attempts similarly failed to recover a stable, fully decompilable .NET assembly.

During dynamic analysis within the FLARE VM environment, the malware repeatedly terminated shortly after execution, even under debugger-assisted conditions. Breakpoints placed on `NtTerminateProcess` confirmed deliberate self-termination behavior consistent with anti-analysis or virtualization-detection logic intended to suppress malicious functionality within monitored environments.

Although full runtime execution and live network traffic could not be successfully observed, static analysis, runtime string extraction, and memory inspection identified multiple indicators associated with:

- HTTP- and SMTP-based communication
- asynchronous networking operations
- encrypted or serialized data handling
- registry interaction
- runtime assembly loading
- potential data exfiltration functionality

Recovered runtime artifacts including `HttpClient`, `PostAsync`, `UploadData`, `CryptoStream`, `JsonConvert`, `Rebex.Net.Smtp`, and `FromBase64String` strongly suggest that the malware are consistent with protected outbound communication functionality and encoded configuration handling despite the inability to fully reconstruct runtime behavior.

These limitations highlight the effectiveness of modern .NET malware protection techniques and reinforce the importance of combining static analysis, debugger-assisted runtime inspection, memory analysis, and behavioral reconstruction techniques during malware triage and incident response investigations.



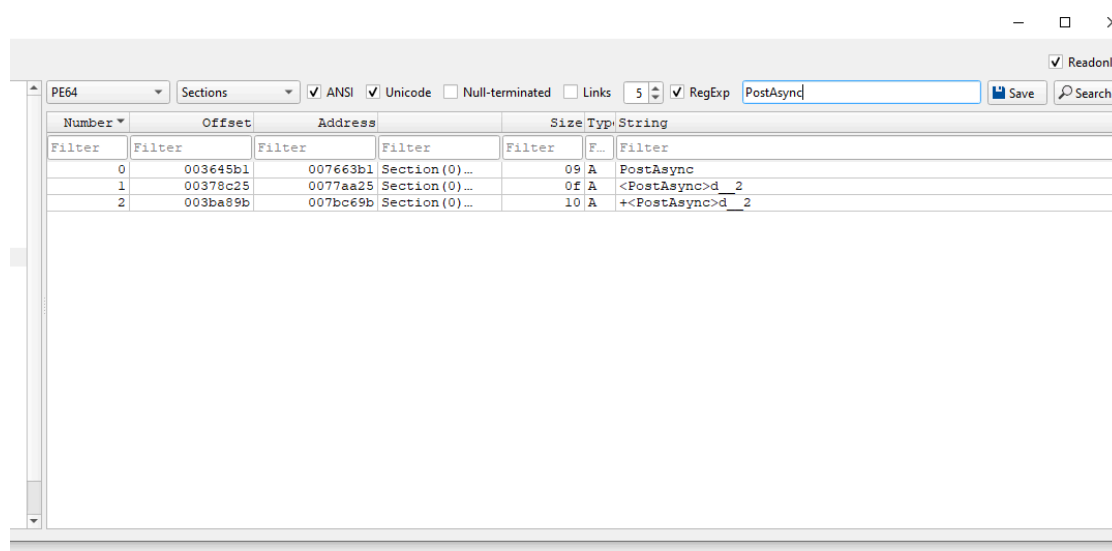
Defense / Anti-Analysis Techniques Identified

The sample employs multiple techniques specifically designed to hinder reverse engineering and analysis:

Technique	Evidence
IL Obfuscation	malformed IL, dnSpy failures
Anti-Debugging	NtTerminateProcess termination behavior
Virtualization Obfuscation	XVM.Runtime indicators
Runtime Assembly Loading	Costura + AssemblyLoader
String Obfuscation	Base64 + CryptoStream + limited plaintext strings
Decompiler Disruption	OverflowException / failed method reconstruction



Packing and Obfuscation Analysis (DIE)



Number	Offset	Address	Size	Type	String
0	003645b1	007663b1	09	A	PostAsync
1	00378c25	0077aa25	0f	A	<PostAsync>d_2
2	003ba89b	007bc69b	10	A	+<PostAsync>d_2

Analysis using Detect It Easy (DIE), runtime string extraction, and memory inspection revealed that the sample is a heavily protected 64-bit .NET executable targeting the .NET Framework (CLR v4.0.30319). The malware contains embedded libraries associated with serialization, networking, and graphical interface functionality, including Newtonsoft.Json and Guna.UI. Runtime artifacts further identified asynchronous networking methods such as **PostAsync**, suggesting support for structured outbound communication during execution.

Heuristic analysis identified multiple defense-evasion and anti-analysis mechanisms, including obfuscation, modified entry point behavior, anti-ILDASM protections, anti-debugging behavior, and anti-dumping techniques. During dynamic analysis, the malware repeatedly invoked **NtTerminateProcess** shortly after launch, further



supporting the presence of runtime protections intended to disrupt debugging and sandbox analysis.

The executable's `.text` section exhibited high entropy and compression characteristics consistent with packing or encrypted resource storage. Additional indicators, including embedded Costura resources, runtime assembly loading behavior, overlay data appended to the executable, and malformed CLR metadata, strongly suggest that the malware dynamically reconstructs or loads protected components in memory at runtime in an effort to hinder static analysis and decompilation.

Runtime Virtualization Indicators

Number	Offset	Address	Size	Type	String
0	003645b1	007663b1	09	A	PostAsync
1	00378c25	0077aa25	0f	A	<PostAsync>d_2
2	003ba89b	007bc69b	10	A	+<PostAsync>d_2

Figure 7. Identified Runtime virtualization-related artifacts associated with XVM protections.

Key Runtime & Embedded Components

The malware contains multiple embedded libraries and runtime components associated with networking, cryptographic operations, serialization, and protected runtime execution. Analysis using Detect It Easy (DIE), runtime string extraction, and

memory inspection identified functionality related to HTTP- and SMTP-based communication, encrypted data handling, asynchronous networking, and runtime assembly loading.

Recovered networking and communication indicators included:

- HttpClient
- GetAsync
- PostAsync
- UploadData
- UploadString
- Rebex.Net.Smtp
- Rebex.Mail
- smtp-mail.outlook.com
- application/json

These artifacts are consistent with networking-related functionality associated with outbound communication and structured data transfer workflows, structured HTTP requests, staged payload delivery, or potential data exfiltration functionality.

Additional serialization and cryptographic indicators identified during runtime analysis included:

- JsonConvert
- DeserializeObject
- CryptoStream
- FromBase64String
- UTF8 / GetString
- System.Security.Cryptography.X509Certificates

These components indicate the malware is capable of processing encoded, serialized, or encrypted configuration and communication data during runtime execution.

The sample also leverages the Costura framework to embed compressed .NET dependencies directly within the executable. Recovered runtime artifacts including:

- Costura
- AssemblyLoader



- ILTemplate

suggest that the malware dynamically loads assemblies from memory rather than dropping additional files to disk, reducing filesystem artifacts and complicating static analysis.

Additional runtime and environment-related indicators identified included:

- v4.0.30319
- mscorlib
- System.Diagnostics
- RegistryKey
- get_RegistryPath
- Local\Temp
- Guna.UI2.WinForms

These artifacts suggest interaction with the Windows runtime environment, registry structures, temporary filesystem locations, and possible graphical user interface components potentially intended to mimic legitimate applications or installers during execution.

Indicators of Compromise

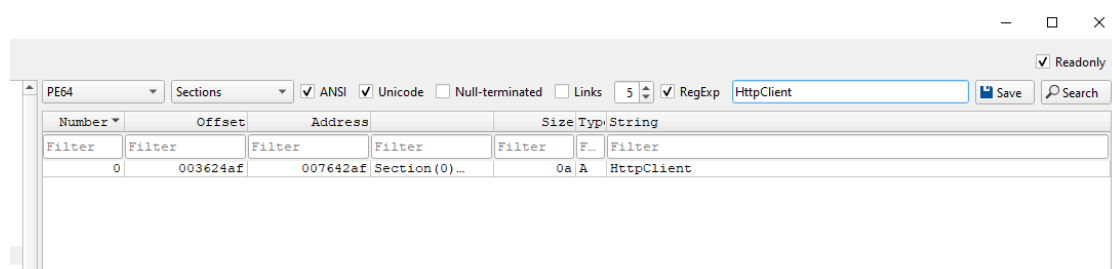
Network Indicators

Number	Offset	Address	Size	Type	String
0	0037d082	0077ee82 Section(0)...	26	A	<DownloadFileWithProgressAsync>b_24_0
1	0037ff60	00781d60 Section(0)...	23	A	<DownloadFileWithProgressAsync>b_1
2	00380da1	00782ba1 Section(0)...	24	A	<DownloadFileWithProgressAsync>d_24
3	003bac75	007bca75 Section(0)...	25	A	+<DownloadFileWithProgressAsync>d_24

Direct outbound network traffic was not successfully observed during dynamic execution within the FLARE VM environment. The malware repeatedly terminated shortly after launch, likely due to anti-analysis or virtualization-detection behavior. Despite the absence of live packet captures, extensive static analysis and runtime



memory inspection identified numerous indicators strongly associated with HTTP- and SMTP-based communication capability.



Recovered networking-related functionality included:

- HttpClient
- HttpResponseMessage
- GetAsync
- PostAsync
- UploadData
- UploadString
- application/json
- Rebex.Net.Smtp
- Rebex.Mail
- smtp-mail.outlook.com

These artifacts strongly suggest support for structured HTTP/S communication and outbound SMTP-based data transmission. The presence of UploadData and UploadString methods suggests support for external data transmission functionality, potentially for command-and-control (C2) communication, staged payload delivery, or data exfiltration.



Number	Offset	Address	Size	Type	String
0	0036344f	0076524f	0c	A	CryptoStream
1	0036348b	0076528b	10	A	CryptoStreamMode

Runtime string analysis additionally identified the presence of **CryptoStream** and **CryptoStreamMode**, indicating support for encrypted data handling and protected communication workflows. These artifacts suggest that outbound traffic, configuration data, or exfiltrated information may be encrypted or encoded during runtime execution to hinder analysis and detection.

Additional serialization and cryptographic indicators identified during runtime string analysis included:

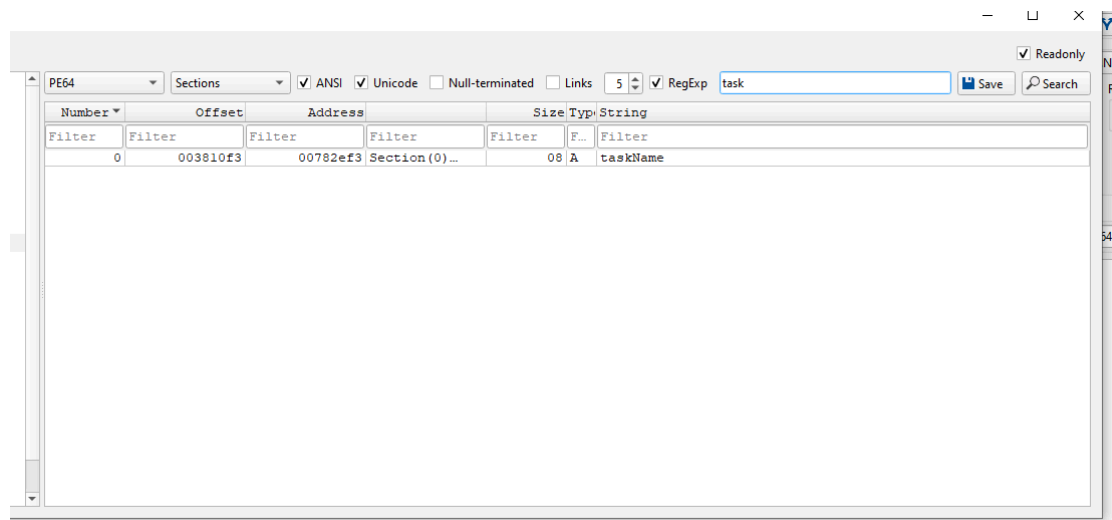
- JsonConvert
- DeserializeObject
- FromBase64String
- UTF8 / GetString
- CryptoStream

These findings suggest that network traffic, configuration data, or exfiltrated information may be encoded, serialized, or encrypted during runtime execution.

Although no live domains or IP addresses were recovered during analysis, the combined networking, serialization, and cryptographic indicators strongly support the conclusion that the malware was designed to communicate with remote infrastructure using protected or obfuscated communication channels.



Host-based Indicators



Host-based analysis identified numerous indicators associated with protected .NET malware loaders and stealth-oriented execution behavior. The analyzed sample is delivered as a 64-bit .NET executable leveraging the Common Language Runtime (CLR) and embeds multiple compressed dependencies internally using the Costura framework.

The executable contains embedded libraries and runtime components associated with networking, serialization, cryptographic handling, and process interaction, including:

- System.Net.Http
- HttpClient
- Newtonsoft.Json
- JsonConvert
- Rebex.Net.Smtp
- Rebex.Mail
- System.Diagnostics
- System.Security.Cryptography
- Guna.UI2.WinForms

These components indicate support for HTTP- and SMTP-based communication, structured data serialization, cryptographic processing, runtime system interaction, and possible graphical interface functionality.



Static and runtime memory analysis additionally identified indicators associated with:

- asynchronous networking operations
- encrypted configuration handling
- runtime Base64 decoding
- registry interaction
- process injection functionality
- environment awareness
- staged payload retrieval

Recovered runtime strings included:

- PostAsync
- GetAsync
- UploadData
- UploadString
- CryptoStream
- FromBase64String
- DeserializeObject
- RegistryKey
- get_RegistryPath
- ProcessInjection
- DriverInject
- DownloadFileWithProgressAsync

The malware also exhibits multiple structural characteristics commonly associated with protected or obfuscated malware, including:

- high-entropy packed sections
- malformed Intermediate Language (IL)
- virtualization-based obfuscation (XVM.Runtime)
- anti-debugging protections
- anti-ILDASM protections
- modified entry point behavior
- runtime assembly loading
- embedded compressed DLL resources

Runtime analysis further revealed repeated self-termination behavior through `NtTerminateProcess` invocation shortly after execution, strongly suggesting the



presence of anti-analysis or virtualization-detection logic intended to suppress execution within monitored environments.

Behavioral / structural indicators identified:

- 64-bit .NET executable using CLR v4.0.30319
- packed and obfuscated executable structure
- runtime assembly loading through Costura
- embedded compressed dependencies
- malformed or virtualized IL code
- anti-debugging and anti-decompilation protections
- encrypted or serialized runtime data handling
- dynamic networking capability
- registry interaction and environment-awareness indicators

Language Identification

Analysis using Detect It Easy (DIE), dnSpy, runtime metadata inspection, and extracted .NET runtime artifacts confirmed that the sample is a managed .NET executable compiled using Microsoft Intermediate Language (MSIL), most likely originating from C#. The presence of asynchronous method naming conventions, .NET runtime libraries, serialization components, and managed runtime structures strongly supports this conclusion.



Rules & Signatures

A full set of YARA rules is included in Appendix A.

A custom YARA rule for this sample is included in Appendix A. The rule is based on static and runtime indicators recovered during analysis, including embedded Costura resources, .NET runtime artifacts, networking and SMTP-related strings, cryptographic and serialization indicators, registry interaction strings, and anti-analysis artifacts.

No confirmed live C2 IP addresses, domains, or URLs were observed during dynamic execution because the sample terminated shortly after launch in the FLARE VM environment. As a result, the included detection logic focuses on host-based and structural indicators rather than network-based signatures.

Key signature components include:

- PE/.NET executable markers
- Costura embedded DLL resource names
- XVM/runtime obfuscation indicators
- HTTP and SMTP communication strings
- JSON serialization indicators
- Base64 and CryptoStream-related strings
- Registry and host-enumeration strings
- Process injection and anti-analysis indicators

These signatures should be treated as medium-confidence detections for an obfuscated Agent Tesla-style .NET loader. They may require additional tuning before use in a production detection environment.



MITRE ATT&CK Mapping

ATT&CK Tactic	Technique	ID	Evidence Identified	Confidence
Defense Evasion	Obfuscated Files or Information	T1027	malformed IL, XVM.Runtime, packed sections, anti-ILDASM	High
Defense Evasion	Software Packing	T1027.002	Costura embedded DLLs, high entropy sections	High
Defense Evasion	Virtualization/Sandbox Evasion	T1497	immediate termination in FLARE VM, NtTerminateProcess behavior	Medium
Command & Control	Application Layer Protocol: Web Protocols	T1071.001	HttpClient, GetAsync, PostAsync, UploadData	High
Exfiltration	Exfiltration Over Web Service	T1567	UploadString, UploadData, application/json	Medium
Exfiltration	Exfiltration Over Alternative Protocol	T1048	Rebex.Net.Smtp, smtp-mail.outlook.com	Medium
Discovery	System Information Discovery	T1082	get_UserName, GetHostName, hwid	High



Discovery	Query Registry	T1012	RegistryKey, get_RegistryPath	Medium
Execution	Shared Modules	T1129	Costura runtime assembly loading	High
Defense Evasion	Reflective Code Loading	T1620	Assembly.Load, runtime DLL loading	Medium
Defense Evasion	Process Injection	T1055	ProcessInjection, DriverInject	Medium
Command & Control	Data Encoding	T1132	FromBase64String, UTF8.GetString	High
Defense Evasion	Deobfuscate/Decode Files or Information	T1140	CryptoStream, Base64 decoding	High

Conclusion

Analysis of the submitted sample identified a heavily obfuscated 64-bit .NET malware loader exhibiting characteristics consistent with Agent Tesla-style information-stealing malware. The executable employed multiple layers of protection and defense-evasion mechanisms, including virtualization-based obfuscation, malformed Intermediate Language (IL), runtime assembly loading, anti-debugging behavior, and protected in-memory execution techniques designed to hinder both static and dynamic analysis.

Although full runtime execution could not be successfully observed within the FLARE VM analysis environment, combined static analysis, runtime memory inspection, string extraction, and debugger-assisted analysis revealed numerous indicators associated with outbound HTTP- and SMTP-based communication, encrypted or serialized configuration handling, runtime assembly reconstruction, registry interaction, possible process injection functionality, and environment-aware execution behavior.



The malware leveraged the Costura framework to embed compressed .NET dependencies directly within the executable and dynamically load required assemblies into memory during runtime. Additional runtime artifacts including HttpClient, PostAsync, Rebex.Net.Smtp, JsonConvert, CryptoStream, and FromBase64String strongly suggest support for protected outbound communication, staged payload handling, and encoded configuration processing.

Repeated invocation of NtTerminateProcess, combined with minimal observable behavior during execution, strongly indicates the presence of sandbox- or virtualization-detection logic intended to suppress malicious activity within monitored environments. The extensive use of packing, virtualization, runtime protection, and anti-analysis techniques demonstrates a deliberate effort to conceal operational functionality and complicate reverse engineering efforts.

Despite the limitations imposed by heavy obfuscation and protected execution behavior, the recovered indicators strongly support the assessment that the sample suggest the sample was intended to operate as a stealth-oriented malware loader supporting in-memory execution, remote communication functionality, and protected runtime data handling. in-memory execution, remote communication, protected configuration handling, and potential credential or system data theft.

This analysis additionally demonstrates the importance of combining static analysis, runtime memory inspection, debugger-assisted analysis, string extraction, and behavioral reconstruction techniques when investigating modern protected .NET malware families.



Appendices

A. Yara Rules

```
rule MAL_DotNet_AgentTesla_Obfuscated_Loader
{
  meta:
    author = "Brianna Wandt"
    description = "Detects obfuscated .NET loader with Agent Tesla-style capabilities, embedded Costura resources, networking, crypto, and anti-analysis indicators"
    date = "2026-04"
    sample_sha256 = "2caae88b894455be6fd124fb2bff81452c7f5249d3afda7daa5567740300b88a"
    malware_family = "Agent Tesla-style"
    confidence = "medium-high"

  strings:
    /* PE / .NET indicators */
    $mz = { 4D 5A }
    $dotnet1 = "mscorlib" ascii wide
    $dotnet2 = "v4.0.30319" ascii wide

    /* Costura embedded dependency indicators */
    $costura1 = "costura." ascii wide
    $costura2 = ".dll.compressed" ascii wide
    $costura3 = "costura.newtonsoft.json.dll.compressed" ascii wide
    $costura4 = "costura.guna.ui2.dll.compressed" ascii wide
    $costura5 = "costura.sharpx.dll.compressed" ascii wide

    /* Obfuscation / runtime protection */
    $xvm1 = "XVM.Runtime" ascii wide
    $anti1 = "NtTerminateProcess" ascii wide
    $anti2 = "Failed to load library" ascii wide
    $anti3 = "Failed to find function" ascii wide
    $anti4 = "DLL name cannot be null or empty" ascii wide

    /* Networking / exfil capability */
    $net1 = "HttpClient" ascii wide
    $net2 = "PostAsync" ascii wide
    $net3 = "GetAsync" ascii wide
    $net4 = "UploadData" ascii wide
}
```



```
$net5 = "UploadString" ascii wide  
$net6 = "application/json" ascii wide  
$smtp1 = "Rebex.Net.Smtp" ascii wide  
$smtp2 = "Rebex.Mail" ascii wide  
$smtp3 = "smtp-mail.outlook.com" ascii wide
```

```
/* Encoding / crypto / serialization */  
$crypto1 = "CryptoStream" ascii wide  
$crypto2 = "CryptoStreamMode" ascii wide  
$crypto3 = "FromBase64String" ascii wide  
$json1 = "JsonConvert" ascii wide  
$json2 = "DeserializeObject" ascii wide
```

```
/* Host / environment awareness */  
$host1 = "GetHostName" ascii wide  
$host2 = "get_UserName" ascii wide  
$host3 = "RegistryKey" ascii wide  
$host4 = "get_RegistryPath" ascii wide  
$host5 = "Win32_BaseBoard" ascii wide  
$host6 = "SerialNumber" ascii wide
```

```
/* Injection / execution indicators */  
$inj1 = "ProcessInjection" ascii wide  
$inj2 = "DriverInject" ascii wide  
$inj3 = "SuccessInject" ascii wide  
$exec1 = "cmd.exe" ascii wide  
$exec2 = "CleanTempAsync" ascii wide
```

condition:

```
$mz at 0 and  
1 of ($dotnet*) and  
(  
  2 of ($costura*) and  
  (  
    2 of ($net*) or  
    1 of ($smtp*) or  
    2 of ($crypto*) or  
    1 of ($json*)  
  ) and  
  (  
    1 of ($xvm*) or  
    2 of ($anti*) or  
    2 of ($host*) or  
    1 of ($inj*)  
  )  
)
```



```
)  
{
```

This YARA rule is designed to detect the analyzed Agent Tesla-style .NET loader based on structural and behavioral indicators recovered during static, runtime string, and memory analysis. The rule combines PE/.NET markers, Costura embedded resource artifacts, networking and SMTP indicators, cryptographic and serialization strings, host-environment awareness artifacts, and anti-analysis indicators. Because several of these strings may also appear in legitimate .NET applications, the rule requires multiple indicator categories to reduce false positives.